When to use PUT and/or POST

If we want to control the IDs or we already know the IDs of our documents, we use the `PUT` method to index the documents. They can be our domain objects with a predefined identity strategy (like primary keys) that we can adhere to. Retrieving documents using an ID may be a reason to consider which way to learn. If we know the document ID, we can use the document APIs to get the document (we discuss retrieval shortly). For example, here we provide an ID for a `movie` document:

```
POST movies/_doc/1
{
  "movie":"The Godfather",
}
```

On the other hand, using IDs for documents originating from streaming data or time-series events doesn't make sense (imagine price quotes originating from a pricing server, share price fluctuations, system alerts from a cloud service, tweets, or heartbeats from an automated car). Having a randomly generated UUID is good enough for these events and messages. But because the IDs are randomly generated, we may need to write a search query to retrieve documents rather than simply retrieving them using IDs as we do for `PUT`s.

In summary, use the HTTP `POST` action to index documents that don't have a business identity. For example, we don't provide an ID for a movie review document:

```
POST movies_reviews/_doc
{
  "review":"The Godfather movie is a masterpiece..",
}
```

When indexing a document, the document index API doesn't care if the document exists or not. If we index it for the first time, the document is created and stored as expected. If we index the same document again, it is saved even if the content is completely different than the earlier document. Does Elasticsearch block the operation of overwriting the document's content? Let's find out.

USING _CREATE TO AVOID OVERRIDING A DOCUMENT

Let's change course slightly and see what happens if we execute the following query.

Listing 5.4 Indexing the incorrect document content

```
PUT movies/_doc/1
{
  "tweet":"Elasticsearch in Action 2e is here!"
}
```

Not a movie but a tweet

In the example, we are indexing a document with a tweet into a `movies` index with a document ID of `1`. Hold on a second: don't we already have a `movie` document (*The Godfather*) with that ID in our store? Yes, we do. Elasticsearch has no control over such overwriting operations. The responsibility is passed down to the application or the user.